

ANDREJ KARPATHY | 中文翻译整理

神经网络训练指南

A Recipe for Training Neural Networks

从理解数据、建立可信基线，到过拟合、正则化、调参和冲刺 SOTA 的完整方法论

“训练神经网络真正需要的，不是快，而是耐心、细致，以及对每一个环节的验证。”

原文作者：Andrej Karpathy

原文链接：<https://karpathy.github.io/2019/04/25/recipe/>

中文整理：根据用户提供的博客全文翻译编排，仅供学习交流

目录

文章先解释两个根本事实，再给出一套从简单到复杂、逐步验证的训练流程。

目录	2
开篇：两个必须接受的事实	3
训练配方	5
1. 与数据融为一体	6
2. 搭建训练评估骨架，建立简单基线	7
3. 先让模型过拟合	10
4. 正则化	12
5. 调参	14
6. 榨干最后一点性能	15
结语	16

开篇：两个必须接受的事实

很多深度学习项目的困难，并不在于把代码运行起来，而在于确认这段代码究竟学到了什么、为什么有效，以及哪里正在悄悄出错。下面两个事实，是整套训练方法的起点。

1. 神经网络训练是一种“会泄漏的抽象”

人们常说，训练神经网络很容易上手。许多库和框架都喜欢展示三十行左右的“奇迹代码”，仿佛只要把数据接进去，问题便会自动解决。常见的示例大概是这样：

```
>>> your_data = # put your awesome dataset here
>>> model = SuperCrossValidator(
...     SuperDuper.fit, your_data, ResNet50, SGD0ptimizer
... )
# conquer world here
```

这类库和示例会唤起我们对传统软件开发的熟悉感：在那个世界里，干净的 API 和可靠的抽象通常是可以实现的。以 [Requests](#) 库为例：

```
>>> r = requests.get(
...     'https://api.github.com/user', auth=('user', 'pass')
... )
>>> r.status_code
200
```

这确实很棒。一个勇敢的开发者替你承担了解查询字符串、URL、GET/POST 请求、HTTP 连接等细节的负担，并把绝大多数复杂性隐藏在几行代码之后。这正是我们熟悉、也习惯期待的软件抽象。

遗憾的是，神经网络完全不是这样。一旦你的任务稍微偏离“训练一个 ImageNet 分类器”，它就不再是可以直接拿来即用的现成技术。我曾在文章 [《是的，你应该理解反向传播》](#) 中专门拿反向传播举例，把它称为一种“会泄漏的抽象”。但实际情况要严重得多。

反向传播加 SGD 不会神奇地让网络自动工作；批归一化不会神奇地让模型更快收敛；RNN 不会让你随手“接入”文本；能够把一个问题写成强化学习，也不意味着你就应该这样做。如果坚持在不了解原理的情况下使用这些技术，你很可能会失败。于是便引出了第二个事实。

2. 神经网络训练会悄无声息地失败

普通代码一旦损坏或配置错误，往往会抛出异常：本来需要字符串的地方传入了整数，函数只接受三个参数，导入失败，某个键不存在，两个列表的元素数量不一致。很多功能还可以通过单元测试来验证。

但在神经网络训练里，这些只是最表层的问题。代码在语法上可能完全正确，所有模块也都能运行，可整个系统的组合方式依然可能是错的，而且非常难以察觉。潜在错误面非常大，错误更多是逻辑性的，而不是语法性的，也很难通过单元测试覆盖。

- 做图像增强时进行了左右翻转，却忘记同步翻转标签。网络仍可能表现得出奇地不错，因为它可以在内部先判断图像是否被翻转，再把自己的预测反向翻转回来。
- 自回归模型因为一个偏移一位的错误，意外地把它正要预测的目标也作为了输入。
- 原本想裁剪梯度，却错误地裁剪了损失，结果训练过程直接忽略了那些异常样本。
- 从预训练检查点初始化了权重，却没有使用原训练过程中的输入均值。
- 正则化强度、学习率、学习率衰减、模型规模等设置不合理。

因此，配置错误的神经网络只有在你运气很好时才会抛出异常。更多时候，它会照常训练，只是悄悄地变差一点。

这一点再怎么强调都不为过：训练神经网络时，“又快又猛”的做法行不通，它只会带来痛苦。更可靠的方式是保持耐心、采取防御性思维、对细节近乎偏执，并尽可能把一切都可视化。

当然，让神经网络真正工作良好，本来就免不了有一些痛苦。但严谨、谨慎、怀疑精神和大量可视化能够显著减少这种痛苦。根据我的经验，在深度学习中最能预测成功的品质，是耐心和对细节的关注。

训练配方

基于前面两个事实，我逐渐形成了一套在神经网络应用于新问题时固定遵循的流程。它非常认真地对待前述原则：始终从简单走向复杂；每一步都先提出对结果的明确假设，再通过实验验证。如果实验不符合预期，就持续调查，直到找到问题。

我们最需要避免的，是一次性引入大量未经验证的复杂性。这样做几乎必然会带来程序错误或配置错误，而且这些问题可能需要很久才能定位，甚至永远也找不到。

如果编写神经网络代码也像训练神经网络，那么你应该使用非常小的“学习率”：每次只做一点改动、提出一个猜测，然后在每轮迭代后评估完整的测试集。

1. 与数据融为一体

训练神经网络的第一步，是完全不要碰神经网络代码，而是先彻底检查数据。这一步至关重要。

我通常会花大量时间，以小时为单位，浏览成千上万个样本，理解它们的分布并寻找规律。幸运的是，人脑非常擅长这件事。我曾因此发现数据中存在重复样本，也曾找到损坏的图像和错误的标签。我会检查数据是否不平衡、是否带有偏差。

与此同时，我还会留意自己是怎样对数据进行分类的，因为这个过程会提示我们以后应该探索什么样的模型结构。可以问自己：

- 非常局部的特征是否已经足够，还是必须理解全局上下文？
- 数据中存在多大程度的变化，这些变化以什么形式出现？
- 哪些变化只是干扰，可以在预处理阶段消除？
- 空间位置是否重要，还是应该通过平均池化把位置信息消除？
- 细节有多重要？图像最多可以下采样到什么程度？
- 标签中有多大噪声？

神经网络本质上可以看成数据集经过压缩和编译后的版本。充分理解数据之后，你才能观察网络的正确或错误预测，并理解这些预测可能来自哪里。如果网络给出一个与你在数据中观察到的规律明显不一致的预测，就说明某个地方出了问题。

获得定性认识之后，最好再写一些简单代码，按照所有能想到的维度搜索、筛选和排序数据，例如标签类型、标注大小、标注数量等，并将各个维度的分布和离群点可视化。离群点尤其重要，它们几乎总能揭示数据质量或预处理中的错误。

核心道理：模型只能学习你真正交给它的数据，而不是你以为自己交给它的数据。

2. 搭建训练评估骨架，建立简单基线

既然已经理解了数据，现在是否可以立刻拿出华丽的多尺度 ASPP、FPN 和 ResNet，开始训练强大的模型？当然不行。那是一条通往痛苦的道路。

下一步应该搭建完整的训练与评估骨架，并通过一系列实验逐渐确认它的正确性。这个阶段最好选一个简单到几乎不可能写错的模型，例如线性分类器或非常小的卷积网络。我们要训练它、可视化损失和准确率等指标、查看模型预测，并围绕明确假设进行一系列消融实验。

固定随机种子

始终使用固定随机种子，保证代码运行两次会得到同样的结果。它消除了一个变化来源，让实验更容易比较，也能保护你的理智。

简化一切

关闭所有当前不必要的复杂功能。例如，这个阶段一定要关闭数据增强。数据增强是一种以后可能采用的正则化策略，但此时它只是又一个可能引入低级错误的地方。

提高评估结果的有效位数

绘制测试损失时，应在整个较大的测试集上执行评估。不要只绘制每个批次的测试损失，然后依靠 TensorBoard 做平滑。我们追求的是正确性，为了保持理智，愿意用更多时间换取可信结果。

验证初始化时的损失

确认训练开始时的损失恰好处在理论上正确的数值。例如，如果最后一层初始化正确，softmax 分类器的初始损失应该约为 $-\log(1 / n_classes)$ 。L2 回归、Huber 损失等也都可以推导出相应的默认值。

做好初始化

正确初始化最后一层的权重和偏置。例如，如果回归目标的均值是 50，就把最后一层偏置初始化为 50。如果数据集中正负样本比例为 1:10，就设置 logits 的偏置，让网络初始化时预测正类的概率约为 0.1。

正确的初始化可以加快收敛，并消除“曲棍球杆”形状的损失曲线。出现这种曲线时，网络在最开始的若干轮中基本只是在学习一个合适的偏置。

建立人类基线

除损失外，还应监控人类可以理解和检查的指标，例如准确率。只要可能，就评估你自己在任务上的准确率并与模型比较。也可以把测试数据标注两次，对每个样本把第一次标注当作预测，第二次标注当作真实答案。

建立与输入无关的基线

训练一个不依赖输入的基线模型，最简单的方法是把所有输入都设为零。它的表现应该差于使用真实输入训练的模型。事实是否如此？换句话说，模型究竟有没有从输入中提取到任何信息？

让模型过拟合一个批次

只用一个包含极少样本的批次进行训练，甚至可以只有两个样本，并让模型过拟合这个批次。为此，可以增加模型容量，例如增加层数或卷积核数量，然后确认损失能够达到理论最低值，例如零。

我还喜欢在同一张图上同时绘制标签和预测，确认达到最低损失后两者能够完美重合。如果不能，系统中一定存在错误，此时不能进入下一阶段。

确认训练损失会随容量增加而下降

由于此时使用的是玩具模型，理想情况下它应该处于欠拟合状态。稍微增加一点模型容量，训练损失是否像预期那样下降了？

在输入网络前的最后一刻可视化数据

可视化数据最明确、最正确的位置，是紧挨着 `y_hat = model(x)` 之前，也就是把真正即将进入网络的原始张量和标签解码并显示出来。在 TensorFlow 中，则是在 `sess.run` 之前。

这里是唯一的“事实来源”。我已经记不清这种检查曾多少次救过我，并揭示出数据预处理或增强中的问题。

可视化预测的变化过程

在训练过程中，持续可视化模型对一个固定测试批次的预测。预测随训练变化的动态，会让你对训练过程形成非常好的直觉。如果预测以某种方式抖动得过于剧烈，你甚至可以感受到网络正在艰难地拟合数据，从而发现训练不稳定。学习率过低或过高，也能从抖动幅度中明显看出来。

利用反向传播绘制依赖关系

深度学习代码经常包含复杂的向量化和广播运算。一个很常见的错误，是错误使用 `view`，而本应使用 `transpose` 或 `permute`，从而无意中混合了批次维度中的信息。令人沮丧的是，网络通常仍然能够正常训练，因为它会学着忽略其他样本的数据。

调试这类问题的一种方法是，把损失设置为一个非常简单的量，例如第 i 个样本所有输出之和，然后一直反向传播到输入，确认只有第 i 个输入具有非零梯度。同样的方法也能检查自回归模型在时刻 t 是否只依赖 1 到 $t-1$ 时刻。更一般地说，梯度告诉你网络中“什么依赖什么”，因此是非常有用的调试工具。

先实现特例，再推广到一般情况

这是一个更普遍的编程建议。人们经常因为一开始就尝试从零实现过于通用的功能而引入错误。我更喜欢先写一个只解决当前具体问题的函数，把它运行正确，然后逐步推广，同时确认结果保持一致。

向量化代码尤其适合这种做法：我几乎总是先写出包含完整循环的版本，再一次消除一层循环，逐步转换成向量化实现。

这一阶段的目标不是得到高分，而是建立一个你真正信任的测量系统。没有可信的训练与评估骨架，后面的每一次提升都可能只是幻觉。

3. 先让模型过拟合

到了这个阶段，我们应该已经充分理解数据集，完整的训练和评估流程也能正常工作。对于任意模型，我们都能以可复现的方式计算出一个值得信赖的指标。

同时，我们还掌握了与输入无关的基线表现、几个简单基线的表现，以及人类在任务上的大致表现。现在终于可以开始反复尝试，寻找一个真正优秀的模型。

我喜欢把寻找好模型分成两个阶段：首先使用容量足够大的模型，让它能够过拟合，也就是专注于降低训练损失；然后再进行适当的正则化，牺牲一部分训练集表现，换取更好的验证集表现。

之所以分成两个阶段，是因为如果无论使用什么模型都无法把训练误差降到足够低，那么系统中很可能仍然存在问题、程序错误或配置不当。

选择模型：不要逞英雄

为了得到较低的训练损失，需要为数据选择合适的模型架构。在模型选择方面，我最重要的建议是：不要逞英雄。

很多人迫不及待地想发挥创造力，把神经网络工具箱里的各种“乐高积木”堆叠成稀奇古怪的架构，因为这些结构在他们看来很有道理。在项目早期，一定要强烈抵制这种诱惑。

直接找到与你的任务最接近的论文，然后复制其中结构最简单、同时又能取得良好效果的模型。例如，如果正在做图像分类，第一次实验就直接使用 ResNet-50。等基线跑通之后，再尝试更定制化的架构并超过它。

Adam 是一个安全的选择

项目早期建立基线时，我喜欢使用 Adam，并把学习率设为 $3e-4$ 。根据我的经验，Adam 对超参数更加宽容，即使学习率设置得不够好，通常也还能工作。

对于卷积神经网络，经过良好调参的 SGD 几乎总能略微超过 Adam，但 SGD 的最佳学习率区间更窄，也更加依赖具体问题。如果使用 RNN 或相关序列模型，Adam 更为常见。项目初期还是不要逞英雄，先遵循最相关论文的做法。

每次只增加一种复杂性

如果分类器需要接收多种信号，建议一次只加入一种。每加入一种信号，都要确认它确实带来了符合预期的性能提升。不要一开始就把所有可能有用的东西全部塞进模型。

也可以沿其他维度逐步增加复杂性，例如先使用较小尺寸的图像，等流程跑通后再增大图像尺寸。

不要相信默认的学习率衰减策略

如果把其他领域的代码改造后用于自己的任务，一定要格外小心学习率衰减。不同问题需要不同的衰减策略；更糟糕的是，常见实现通常根据当前 epoch 数决定衰减，而 epoch 的实际意义会随数据集大小发生巨大变化。

例如，ImageNet 训练可能会在第 30 个 epoch 把学习率降为原来的十分之一。但如果你训练的不是 ImageNet，几乎肯定不应该直接照搬这个设置。否则代码可能在你不知情的情况下过早地把学习率降到接近零，使模型还没来得及收敛就停止有效学习。

在我自己的工作中，我通常会完全关闭学习率衰减，先使用恒定学习率，直到整个项目的最后阶段才调整衰减策略。

先证明模型有能力记住训练集，再讨论如何让它泛化。否则，你甚至无法区分“模型能力不足”和“实现存在错误”。

4. 正则化

理想情况下，现在我们已经有了一个容量足够大的模型，并且它至少能够很好地拟合训练集。接下来需要进行正则化：牺牲一部分训练集准确率，换取更高的验证集准确率。

获取更多真实数据

在几乎所有实际场景中，正则化模型最好、最优先的方法，都是增加更多真实训练数据。一个常见错误是花费大量工程时间，试图从很小的数据集中榨取更多性能，而这些时间原本可以用于收集更多数据。

据我所知，对于一个配置合理的神经网络，增加数据几乎是唯一一种能够在相当长时间内持续、稳定提升性能的方法。另一种方法是模型集成，前提是能承担计算成本，不过集成收益通常会在大约五个模型之后趋于饱和。

数据增强

仅次于真实数据的是“半假数据”，也就是尝试更激进的数据增强。

创造性的数据扩充

如果“半假数据”还不够，完全生成的“假数据”也可能有帮助。人们已经发明了很多创造性方法，例如领域随机化、使用模拟环境、将真实或模拟数据插入场景的混合方法，以及使用 GAN 生成数据。

使用预训练模型

只要条件允许，使用预训练网络通常都不会有坏处，即使你已经拥有足够多的数据。

优先坚持监督学习

不要对无监督预训练过度兴奋。不管 2008 年的博客文章怎么说，据我所知，当时各种无监督预训练方法在现代计算机视觉中都没有取得特别强的结果。不过 NLP 使用 BERT 及其相关模型取得了不错的效果，这可能是因为文本具有更强的人工组织性，而且信噪比更高。

降低输入维度

移除可能包含虚假相关信号的特征。当数据集很小时，每增加一个无关或虚假的输入信号，就相当于给模型增加一次过拟合的机会。类似地，如果图像中的底层细节并不重要，可以尝试输入更小尺寸的图像。

减小模型规模

在很多情况下，可以利用领域知识给网络施加约束，从而减小模型规模。例如，过去人们经常在 ImageNet 骨干网络顶部使用全连接层，后来这些层逐渐被简单的全局平均池化替代，从而消除了大量参数。

减小批次大小

由于批归一化内部使用批次统计量，减小批次大小在某种程度上相当于增强正则化。小批次计算出的均值和标准差只是总体真实值的粗略估计，因此归一化的缩放与偏移会产生更多波动，相当于给训练过程加入额外噪声。

加入 Dropout

对于卷积神经网络，可以使用 Dropout2d，也叫空间 Dropout。不过要谨慎、适量地使用，因为 Dropout 似乎与 批归一化 配合得不太好。

增大权重衰减

增大 weight decay 的惩罚强度。

提前停止

根据验证集损失决定何时停止训练，争取在模型即将开始明显过拟合时把它停下来。

尝试更大的模型

这一点放在最后，并且只建议配合提前停止使用。虽然更大的模型最终会过拟合得更加严重，但在合适时机提前停止后，它们的表现往往会明显优于较小模型。

最后，为了进一步确认网络确实是一个合理的分类器，可以把第一层权重可视化。对于图像模型，正常情况下应该看到清晰而合理的边缘检测器。如果第一层卷积核仍像随机噪声，系统中可能存在问题。网络内部激活有时也会呈现奇怪的伪影，从而提示潜在错误。

5. 调参

现在，你应该已经真正进入实验循环：围绕数据集探索广阔的模型空间，寻找能够取得较低验证损失的架构。

优先使用随机搜索，而不是网格搜索

同时调整多个超参数时，网格搜索看起来很有吸引力，因为它似乎能够系统覆盖所有组合。但实际上，[随机搜索通常是更好的选择](#)。

直观上，这是因为神经网络对某些超参数非常敏感，对另一些参数却可能完全不敏感。在极端情况下，假设参数 a 很重要，而改变参数 b 几乎没有效果。与其在少数几个固定的 a 值上反复尝试不同的 b ，不如随机采样更多不同的 a ，更充分地探索真正重要的参数。

超参数优化

目前有很多复杂的贝叶斯超参数优化工具，我的一些朋友也报告说它们效果不错。不过根据我的个人经验，探索广阔的模型和超参数空间时，当前最先进的方法还是：找一个实习生来调。

开玩笑的。

调参的前提是实验本身可信。随机搜索解决的是“如何更有效地探索”，它无法修复错误的数据、指标或训练流程。

6. 榨干最后一点性能

找到表现最好的模型架构和超参数之后，还可以使用一些额外技巧，把系统中最后一点性能榨出来。

模型集成

模型集成几乎是一种可以保证提升性能的方法，在很多任务上通常能额外提高大约 2% 的准确率。如果测试阶段无法承担多个模型的计算成本，可以研究如何利用 [暗知识](#)，把集成模型蒸馏成一个单独的神经网络。

让它继续训练

我经常看到人们发现验证损失开始趋于平稳后，就忍不住停止训练。但根据我的经验，神经网络保持有效训练的时间往往长得超出直觉。

有一次，我在寒假期间不小心让一个模型一直训练。等到一月份回来时，它已经达到了 SOTA，也就是“当前最先进水平”。

结语

走到这里，你就已经具备了成功所需的全部要素。

- 深入理解了技术、数据集以及需要解决的问题；
- 建立了完整的训练和评估基础设施；
- 通过充分检查，对整个系统的正确性建立了高度信心；
- 逐步探索了越来越复杂的模型；
- 每一次性能提升基本都符合预期，并且能够解释它来自哪里。

现在，你已经准备好阅读大量论文、开展大量实验，并取得属于自己的 SOTA 结果了。

整篇文章的核心并不是某个优化器或某种架构，而是一种实验纪律：
从简单开始；一次只改变一个因素；在每一步之前写下预期；如果结果不符合预期，就先找原因，不要继续堆复杂性。

祝你好运！

原文：Andrej Karpathy, A Recipe for Training Neural Networks
<https://karpathy.github.io/2019/04/25/recipe/>